



University of Colombo, Sri Lanka

University of Colombo School of Computing

BACHELOR OF SCIENCE IN INFORMATION SYSTEMS

Second Year Examination - Semester I - UCSC AY22 [October 2025]

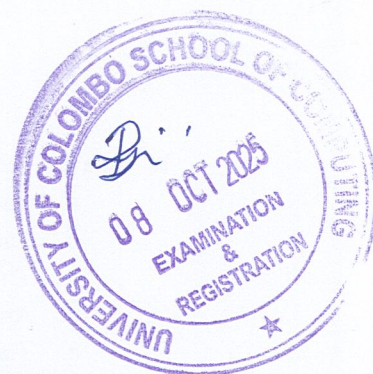
IS2202 — Advanced Data Structures and Algorithms

(Two (2) Hours)

Answer ALL questions

Number of Pages = 12

Number of MCQs = 35; Structured Questions = 1



To be completed by the candidate							
Index Number:							

097

Important Instructions to candidates:

- I. This is a **closed book exam**.
- II. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
- III. This paper has two (02) parts, Part I and Part II. **Part I consists of thirty-five (35) MCQ questions** for seventy (70) marks. Select the best suitable answer(s) among the given five (5) choices. **Part II** consists of one **Structured Question** for thirty (30) marks.
- IV. MCQ should be marked on the **MCQ answer sheet** and the answers to the **Structured question in Part II need to be written on this paper**.
- V. Write your **index number** on the answer sheets.
- VI. Answer **ALL** questions.
- VII. Each MCQ question has five (05) answers with one (01) or more (up to 05) correct answers. Each MCQ question is worth two (02) marks. Suppose a question has two (02) correct answers and three (03) incorrect answers. **+2 marks will be shared between the correct options** (1 mark per option), and **-2 marks will be shared between incorrect options** (-0.67) per option. If one correct and incorrect answer is marked, the final mark for the question will be $1 + (-0.67) = 0.33$.
- VIII. Calculators, Mobile phones, Smart watches or any electronic device capable of storing and retrieving data are **not allowed**.

To be completed by the examiners

1	
2	
3	
4	
Total	

PART I

1. In a B-Tree of order m :
 - a. Each node can have at most $m-1$ keys
 - b. Each node can have at most m children
 - c. The root must always have at least 3 children
 - d. An internal node must have at least $\lceil m/2 \rceil$ children
 - e. The root can have only 1 child if not a leaf
2. In a B-Tree of order 5, the maximum number of keys a node can have is:
 - a. 3
 - b. 4
 - c. 5
 - d. 6
 - e. 7
3. Which is a property of leaf nodes in a B-Tree?
 - a. They may exist at different depths
 - b. All leaf nodes exist at the same level
 - c. They contain between $\lceil m/2 \rceil - 2$ and $m-2$ keys
 - d. Leaf nodes have no children
 - e. Leaf nodes must always be full
4. When inserting into a B-Tree:
 - a. Keys are inserted in sorted order in nodes
 - b. A node splits if it exceeds maximum keys
 - c. The median key moves up to parent on split
 - d. Splitting only happens at the root
 - e. New nodes are added at the bottom level
5. Which operations in B-Trees take $O(\log n)$ time?
 - a. Search
 - b. Insert
 - c. Delete
 - d. Traverse
 - e. Update (Merge + Reshuffle)
6. A B-Tree of order 5 has:
 - a. Root: minimum 1 key, maximum 4 keys
 - b. Internal nodes: minimum 2 keys, maximum 4 keys
 - c. Leaf nodes: 0 children
 - d. Internal nodes: at least 3 children
 - e. Root can have 0 keys if it is a leaf
7. Which deletion cases are valid in B-Trees?
 - a. Leaf node deletion (direct removal)
 - b. Borrowing from sibling if possible
 - c. Merging with sibling if borrowing not possible
 - d. Replacing internal node key with inorder predecessor/successor
 - e. Ignoring minimum key condition for balance

8. Which properties hold for a B+ Tree of order m ?
- Each node can have maximum $m-1$ keys
 - Each internal node must have at least $\lceil m/2 \rceil$ children
 - Leaf nodes must have at least $\lceil m-1/2 \rceil$ keys
 - Root must always have at least 3 children
 - Internal nodes can hold actual data values
9. In B+ Tree insertion, if the parent node is also full after a split:
- Split the parent node as well
 - Move the leftmost key to the new right node
 - Convert to a B-Tree structure
 - Stop insertion and discard the key.
 - Continue splitting upwards if needed
10. Which of the following differentiates B+ Trees from B-Trees?
- B+ Trees store data only at leaf nodes
 - Internal nodes contain only keys and pointers
 - Leaf nodes are linked for range queries
 - Internal nodes contain data records
 - B+ Trees are slower for range queries
11. Huffman codes are:
- Always fixed-length
 - A type of prefix code
 - Optimal variable-length codes
 - Used for lossless compression
 - Guaranteed to save 100% space
12. In Huffman tree construction?
- The tree has exactly n leaves for n characters
 - Internal nodes = $n - 1$
 - Every non-leaf has 2 children (full binary tree)
 - The most frequent character has the longest codeword
 - The least frequent characters are deeper in the tree
13. The Huffman algorithm uses:
- A maximum priority queue
 - A minimum priority queue
 - Character frequencies as weights
 - A greedy approach
 - Divide and conquer strategy
14. What is the time complexity of Huffman coding?
- $O(n)$
 - $O(\log n)$
 - $O(n \log n)$
 - $O(n^2)$
 - $O(1)$

15. Run-Length Coding (RLC) works best for:

- a. Complex images with many colors
- b. Binary or two-valued images
- c. Repeated patterns of pixels
- d. Highly random data
- e. Preprocessed images with reduced gray levels

16. Typical compression ratios of bitplane RLC with Gray code preprocessing are:

- a. typically achieves 0.5 – 1.2 for complex 8-bit monochrome images
- b. 10% – 15% improvement over basic RLC
- c. Always lossless when combined with thresholding
- d. Higher for simple graphics files
- e. Reduced when using thresholding

17. What are the key properties required for a problem to be solved using a greedy algorithm?

- a. Greedy-choice property
- b. Optimal substructure
- c. Dynamic programming compatibility
- d. Polynomial time complexity
- e. Recursive decomposition

18. Which of the following problems can be solved optimally using a greedy strategy?

- a. Activity selection
- b. Fractional knapsack
- c. Huffman coding
- d. 0/1 Knapsack
- e. Traveling salesman problem

19. What is the role of $A[i,j] \neq -\text{INF}$ in the recursive selector?

```
if (A[i,j] != -INF) return A[i,j]
```

- a. Memoization
- b. Base case
- c. Check compatibility
- d. Avoid recomputation
- e. Select optimal activity

20. What is the purpose of this pseudo-code?

```
f table[hash] is empty then  
    insert key at table[hash]
```

- a. Insert without collision
- b. Initialize bucket
- c. Handle overflow
- d. Resize table
- e. Delete key

21. What does this pseudo-code represent?

$\text{hash} \leftarrow (\text{h1}(\text{key}) + i \times \text{h2}(\text{key})) \bmod \text{table_size}$

- a. Double hashing
- b. Open addressing
- c. Linear probing
- d. Chaining
- e. Coalesced hashing

22. Which pseudo-code represents linear probing in open addressing?

- a. $\text{hash} \leftarrow (\text{hash} + 1) \bmod \text{table_size}$
- b. $\text{hash} \leftarrow \text{key} \bmod \text{table_size}$
- c. $\text{hash} \leftarrow (\text{key} + i^2) \bmod \text{table_size}$
- d. $\text{hash} \leftarrow (\text{h1}(\text{key}) + i \times \text{h2}(\text{key})) \bmod \text{table_size}$
- e. $\text{hash} \leftarrow \text{key} / \text{table_size}$

23. Which pseudo-code represents coalesced chaining?

- a. $\text{table}[\text{hash}].\text{next_index} \leftarrow \text{next_available_index}$
- b. $\text{table}[\text{hash}] \leftarrow \text{key}$
- c. $\text{table}[\text{next_available_index}] \leftarrow \text{last_n_digits}(\text{key})$
- d. $\text{table}[\text{hash}].\text{next} \leftarrow \text{key}$
- e. $\text{table}[\text{hash}] \leftarrow \text{NULL}$

24. Which of the following statements about open addressing are correct?

- i. All elements are stored directly in the hash table.
 - ii. Load factor must be less than or equal to 1.
 - iii. Open addressing uses pointers to linked lists.
- a. i only
 - b. i and ii
 - c. ii and iii
 - d. i and iii
 - e. All three

25. Which of the following statements about the multiplication method are correct?

- i. It multiplies the key by a constant A.
 - ii. It is more efficient than the division method in all cases.
 - iii. It extracts the fractional part of the product.
- a. i only
 - b. i and ii
 - c. ii and iii
 - d. i and iii
 - e. All three

26. Which of the following statements about the folding method are correct?

- i. It multiplies the parts to get the hash value.
 - ii. It divides the key into parts.
 - iii. It adds the parts together.
- a. i only
 - b. i and ii
 - c. ii and iii
 - d. i and iii
 - e. All three

27. Consider the following pseudo-code for jump Search. Which of the following are correct?

```
function jumpSearch(arr, target):  
    step ← floor(sqrt(length(arr)))  
    prev ← 0  
    while arr[min(step, length(arr))-1] < target:  
        prev ← step  
        step ← step + floor(sqrt(length(arr)))
```

- a. It performs block-wise jumps
- b. It works on unsorted arrays
- c. It uses $\sqrt{n}$ as optimal jump size
- d. It performs binary search after jumping
- e. It is faster than linear search for large sorted arrays

28. Consider the following pseudo-code. Which of the following are correct?

```
while low ≤ high and target ≥ arr[low] and target ≤ arr[high]:  
    pos ← low + ((target - arr[low]) × (high - low)) /  
    (arr[high] - arr[low])  
    if arr[pos] == target:  
        return pos
```

- a. It checks bounds before probing
- b. It uses interpolation formula
- c. It always returns -1
- d. It works on sorted arrays
- e. It uses jump search

29. Consider the following search algorithm. Which of the following are correct?

```
function searchFunction(arr, target):  
    low ← 0  
    high ← length(arr) - 1  
    while low ≤ high:  
        mid ← (low + high) / 2  
        if arr[mid] == target:  
            return mid
```

- a. The array must be sorted
- b. The search space is halved each iteration
- c. It uses linear search internally
- d. It always returns -1
- e. It performs $O(n)$ comparisons

30. Consider the following pseudo-code for Iterative Activity Scheduling. Which of the following are correct?

```
function itr_activity_sched(s, f):  
    A ← {a[1]}  
    k ← 1  
    for m from 2 to n:  
        if s[m] ≥ f[k]:  
            A ← A ∪ {a[m]}  
            k ← m
```

- a. It builds the solution iteratively
 - b. It checks compatibility using start and finish times
 - c. It uses recursion
 - d. It selects overlapping activities
 - e. It updates the last selected activity
31. Which of the following are examples of asymptotic behavior?
- a. $f(x)=1/x$
 - b. $f(x)=\sin(x)$
 - c. Cutting a pizza into infinite slices
 - d. Multiplying by a constant
 - e. Approaching a limit without reaching it
32. Which of the following functions are unbounded?
- a. $y=\sin(x)$
 - b. $y=e^x$
 - c. $f(x)=x^3$
 - d. $f(x)=1/x$
 - e. $y=\log(x)$
33. Which of the following are true about Big O notation?
- a. It ignores constant factors
 - b. It focuses on large input sizes
 - c. It gives exact runtime
 - d. It describes best-case performance
 - e. It helps compare algorithm efficient
34. What is the time complexity of this function?

```
function binarySearch(arr, target):  
    low ← 0  
    high ← length(arr) - 1  
    while low ≤ high:  
        mid ← (low + high) / 2  
        if arr[mid] == target:  
            return mid  
        else if arr[mid] < target:  
            low ← mid + 1  
        else:  
            high ← mid - 1
```

- a. $O(\log n)$
- b. $O(n)$
- c. $O(n^2)$
- d. $O(1)$
- e. $O(n!)$

35. What is the space complexity of this function?

```
function printPairs(arr):  
    for i from 0 to length(arr) - 1:  
        for j from 0 to length(arr) - 1:  
            print(arr[i], arr[j])
```

- a. $O(n^2)$
- b. $O(n)$
- c. $O(1)$
- d. $O(\log n)$
- e. $O(n!)$

Part II

Question 1 – [30 Marks]

Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores vertices in layers, starting from a source node. It is widely used for shortest path calculations, connectivity checks, and bipartite graph detection.

Q1. a. Explain the working principle of Breadth-First Search and its key data structures.

[10 Marks]

ANSWER BOX

--	--	--

Q1. b. Describe how BFS is used to determine the shortest path in an unweighted graph.

[06 Marks]

ANSWER BOX

Q1. c. Analyze the time and space complexity of BFS.

[04 Marks]

ANSWER BOX

Q1. d. Fill in the blanks and complete the pseudocode for BFS traversal.

[06 Marks]

ANSWER BOX

```
Function BFS(graph, source):  
    For each vertex u in graph:  
  
        _____  
        d[u] ← ∞  
        π[u] ← NULL  
  
    color[source] ← GRAY  
    d[source] ← 0  
    π[source] ← NULL  
    Q ← {source}  
  
    While Q is not empty:  
        u ← DEQUEUE(Q)  
        For each v in Adj[u]:  
            If color[v] == WHITE:  
  
                _____  
                d[v] ← d[u] + 1  
                π[v] ← u  
  
                _____
```

Q1. e. Describe how BFS can be used to check if a graph is bipartite.

[04 Marks]

ANSWER BOX
